

The Interview Challenge

Mobile & Backend Implementation Guide (Version 1)

Purpose

This document defines the first implementation of the Interview Challenge Platform.

The goal is to reuse the existing AI Interview infrastructure while introducing a new entity called **Challenge**, allowing creators to publish interview sessions for recruitment or public competitions.

The feature should extend the current interview system instead of replacing it.

Existing Infrastructure

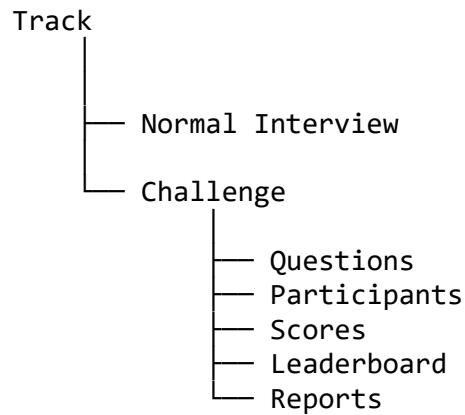
The application already supports:

- Interview Tracks
 - Android
 - Flutter
 - Frontend
 - Backend
 - Data Science
 - etc.
- AI Interview Session
- AI Question Generation
- AI Answer Evaluation
- Interview Report

The Challenge feature should reuse all of the above.

Instead of a user selecting a track and immediately starting an interview, the user joins a Challenge that references one of the existing tracks.

High-Level Architecture



The interview engine remains unchanged.

The Challenge module becomes a wrapper around the existing interview flow.

Backend Responsibilities

The backend becomes responsible for Challenge management.

New Entities

Challenge

Contains:

- Challenge ID
- Creator ID
- Track ID
- Challenge Type
- Visibility
- Status
- Title
- Description
- Difficulty
- Duration
- Start Date
- End Date
- Entry Coin Cost
- Max Attempts

- Max Participants (optional)
 - Invite Code
 - Invite Link (optional)
 - Created Date
-

Challenge Participant

Contains

- Challenge ID
 - User ID
 - Join Date
 - Attempt Number
 - Interview Session ID
 - Overall Score
 - Status
 - Completion Date
 - Rank
-

Challenge Invitation

Private only

Contains

- Challenge ID
 - Phone Number
 - Invitation Status
-

Challenge Leaderboard

Generated from participant results.

Contains

- Rank
 - User
 - Score
 - Completion Time
-

API Groups

The backend should expose APIs for:

Challenge Management

- Create Challenge
 - Edit Challenge
 - Archive Challenge
 - Duplicate Challenge
 - Delete Challenge
-

Public Challenges

- Get Public Challenges
 - Search Challenges
 - Filter Challenges
 - Join Challenge
 - Pay Entry Coins
-

Private Challenges

- Join by Code
 - Join by Link
 - Validate Phone Number
 - Validate Invitation
-

Challenge Progress

- Start Interview
- Resume Interview
- Submit Interview
- Get Interview Report

These APIs should internally call the existing interview engine.

Results

Creator APIs

- Get Participants
- Get Rankings
- Get Reports
- Export Results (optional)

Participant APIs

- My Result
 - My Ranking
 - My History
-

Mobile Responsibilities

The mobile application should be divided into three feature modules.

Module 1

Challenge Discovery

Public Challenges Screen

Displays

- Featured Challenges
- New Challenges
- Trending Challenges
- Track Challenges
- Recently Added

Each card contains

- Title
- Creator
- Track
- Difficulty
- Participants
- Entry Cost
- Estimated Duration

Users can

- Search
 - Filter
 - Join
-

Challenge Details Screen

Displays

- Description
- Track
- Rules
- Entry Cost
- Time Limit
- Difficulty
- Number of Participants
- Creator

Buttons

- Join
 - Share
 - Report
-

Module 2

Challenge Creation

Create Challenge Screen

Creator selects

- Track
- Challenge Type
- Public / Private

Then configures

Basic Information

- Title

- Description

Interview

- Number of Questions
- Difficulty
- Duration

Access

Public

- Entry Coins (Estimated, then server decides)
- Start Date
- End Date

Private

- Phone Numbers
 - Max Attempts
 - Expiration
-

Invite Screen

After creation

Display

- Invitation Code
- Invitation Link

Actions

- Copy
 - Share
-

Allowed Participants Screen

Private only

Creator can

- Add phone numbers
- Remove phone numbers
- Search phone numbers

Module 3

Challenge Dashboard

Creator sees

Overview

- Total Participants
- Completed
- In Progress
- Average Score

Participants List

Each participant

- Name
- Score
- Status

Selecting a participant opens

- AI Report
 - Question Breakdown
 - Coaching Tips
-

Candidate Flow

Public

Open Challenge

↓

Read Details

↓

Spend Coins

↓

Interview

↓

AI Report

↓

Leaderboard

Private

Invitation

↓

Validate

↓

Interview

↓

Creator Reviews Results

Reusing Existing Interview System

No new interview engine should be developed.

Instead,

Challenge creation simply configures an interview before users enter it.

The backend should internally perform:

Challenge

↓

Track

↓

Generate Questions

↓

Interview Session

↓

Evaluation

↓

Interview Report

This keeps all AI logic centralized.

Public Challenge Discovery

Every user should see **all** public challenges regardless of their selected track.

However, challenges matching the user's preferred track should receive greater visual emphasis.

Examples:

- Display first in the list.
- Include a "Recommended for You" section.
- Highlight with a "Matches Your Track" badge.
- Increase recommendation priority in search results.

This encourages exploration while still personalizing the experience.

Challenge Creation

Creators may create challenges for **any** available track.

A Flutter developer may create an Android challenge if desired.

There is no restriction based on the creator's preferred track.

Joining Challenges

Users may join challenges from **any** track.

There are no restrictions based on profile information.

The preferred track only affects recommendations and UI presentation.

Navigation

New Bottom Navigation Bar Item

Challenges

Inside

- Public
- My Challenges
- My Results

Creator actions

- Create Challenge
 - Dashboard
-

Notifications (Future)

Push notifications are skipped for now but can be supported for:

- Invitation received
 - Challenge starting soon
 - Challenge ending soon
 - New participant joined
 - Challenge completed
 - New ranking achieved
-

Development Principles

1. Reuse the existing interview engine whenever possible.
2. Introduce Challenge as a new domain object instead of modifying interview logic.
3. Keep interview generation and evaluation independent from challenge management.
4. Separate creator and participant experiences.
5. Build APIs that support future web and desktop clients.
6. Ensure all business rules (coins, subscriptions, invitations, permissions, rankings) are enforced on the backend.
7. Design the data model to support future challenge types without requiring changes to the interview engine.